

实验四

实验四：中间代码生成

姓名：邵与乔 学号：231240019 日期：2026年6月7日

1. 实验目的

- 在前三个实验（词法分析、语法分析、语义分析）的基础上，将 AST 转换为**三地址码（TAC）中间表示**
- 将中间代码进一步翻译为 **MIPS32 汇编代码**，可在 SPIM 模拟器中运行
- 支持结构体、数组、函数调用等复杂语言特性的代码生成

2. 实验内容

模块	文件	功能
中间代码生成	ir.c / ir.h	遍历 AST，生成三地址码链表，并输出 MIPS32 汇编
词法/语法/语义	复用前三次实验	lexical.l, syntax.y, semantic.c, ast.c 等
主程序	syntax.y 中	依次调用词法→语法→语义→中间代码生成

3. 实现细节

3.1 三地址码中间表示

中间代码以链表形式组织，每条指令为一个字符串，主要包括以下类型：

```
x := y op z    // 二元运算（算术/关系）
x := y         // 简单拷贝
x := #c        // 常量赋值
x := &y        // 取地址
x := *y        // 内存加载
*x := y        // 内存存储
x := CALL f    // 函数调用
IF x relop y GOTO L // 条件跳转
GOTO L         // 无条件跳转
RETURN x       // 返回
```

ARG x // 参数传递
READ x / WRITE x // 内置 I/O
DEC x size // 声明聚合类型变量

3.2 MIPS32 汇编生成

主要流程：

1. **解析 IR**：遍历三地址码链表，识别函数边界（FUNCTION 指令）
2. **变量收集**：分析参数、局部变量、临时变量，计算栈帧偏移
3. **栈帧布局**：

- 参数通过 \$a0-\$a3 传递（超过4个的压栈）
- 局部变量以 \$fp 相对寻址
- 函数序言保存 \$ra、\$fp，结尾恢复

1. **指令翻译**：将每条 TAC 直接映射到 MIPS 指令

内置函数实现：

- read：输出提示，读取整数，返回在 \$v0
- write：输出整数和换行，返回 0

3.3 关键特性支持

特性 **实现方式**

数组 计算元素偏移量： $\text{base} + \text{index} * \text{elemSize}$

结构体 编译时计算各字段偏移，通过 $\text{base} + \text{offset}$ 访问

聚合拷贝 按字逐次拷贝（struct/array 赋值）

聚合传参 传递地址而非值

短路求值 AND/OR 展开为条件跳转标签链

函数声明 只生成声明对应符号的函数定义才输出完整代码

4. 代码结构

```
task4/Lab/Code/  
├── Makefile  
├── lexical.l      # Flex 词法规则  
├── syntax.y       # Bison 语法规则 + main 入口  
├── ast.h / ast.c  # AST 定义与操作  
├── semantic.h / semantic.c # 语义分析  
├── ir.h / ir.c    # 中间代码生成 + MIPS 输出[NEW]  
└── utils.h
```

编译与运行：

`make`

`./parser ../Test/test1.cmm # 输出 MIPS 汇编到 stdout / 文件`

5. 实验总结

难点

解决方案

数组/结构体的内存布局计算	在 IR 层面记录类型信息，编译时计算字段偏移和元素大小
聚合类型的赋值与传参	赋值时按字逐次拷贝；传参时传递地址指针
短路求值的正确翻译	AND/OR 展开为条件跳转标签链，保留短路语义
MIPS 调用约定	前4个参数用 \$a0-\$a3，其余压栈，使用 \$fp 管理栈帧